Project 18 | Mini Compiler: Full Pipeline

Theory Concept	DFA (Lexer) + CFG (Parser) + Code Generation — Full TOC Integration
Application Area	LLM / AI + Security — How Every Programming Language is Built
Language	Python 3
Tools	Python, VS Code, draw.io (all diagrams), optionally PLY (Python Lex-Yacc)
Team Size	4 Students | Duration: 2 Weeks

Project Description

This capstone project ties together the entire Theory of Computation syllabus by building a complete mini compiler for a tiny language. Stage 1: a DFA-based lexer tokenizes source code. Stage 2: a CFG-based recursive descent parser builds a parse tree. Stage 3: a simple code generator walks the parse tree and produces Python output. Students see, end-to-end, how automata theory directly powers every tool they use daily — from Python's own interpreter to AI code generators like GitHub Copilot. This is the most advanced project and is recommended for the strongest group.

Objectives

- Design a mini language with variables, if, while, and expressions
- Build a DFA-based lexer that produces a token stream
- Build a CFG-based parser that produces a parse tree (AST)
- Build a code generator that converts AST to Python code
- Run 10 mini-language programs end-to-end through the full pipeline

2-Week Plan & Member Roles

Week 1 — Theory & Design

- Define mini language syntax
- Write CFG grammar + token list
- Build and test DFA lexer
- Build and test CFG parser + AST generation
- Trace 2 programs through all stages by hand

Week 2 — Code & Demo

- Build code generator (AST → Python)
- Run 10 programs end-to-end
- Display pipeline stages: tokens → AST → output
- Handle 5 error cases gracefully
- Final demo + slides covering all TOC concepts used

Member 1	Member 2	Member 3	Member 4
Language design + DFA lexer	CFG parser + AST code	Code generator (AST → Python)	End-to-end testing + pipeline visualizer + slides

Sample Input / Output

Input:

```
Mini-language program:
  x = 10
  y = 3
  if x > y then
    z = x + y
  end
  print z
```

Output:

```
--- STAGE 1: TOKENS ---
  ID(x) ASSIGN NUM(10) NEWLINE ID(y) ASSIGN NUM(3) ...

--- STAGE 2: PARSE TREE (AST) ---
  Program → [Assign(x,10), Assign(y,3), If(GT(x,y), Assign(z,Plus(x,y))), Print(z)]

--- STAGE 3: GENERATED PYTHON CODE ---
  x = 10
  y = 3
  if x > y:
      z = x + y
  print(z)

--- OUTPUT ---
  13
```

Final Deliverable

A complete 3-stage mini compiler that takes source code in the mini language, shows each pipeline stage, and produces working Python output.